algorithms homework 5

mahmood

December 26, 2022

homework on proving time complexity

Question 1. a pair (A, α) of an array `A` of integers and an integer α is called big-sorted if and only if all items in `A` that are bigger than α exist in the array in a sorted manner, with no importance to the order of items that are smaller or equal to α, for example $([1, 8, 3, 10, 2, 12, 1, 14, 0, 5], 7)$ is big-sorted
prove that there isnt a function that receives an array `B` of integers and an integer `b` that performs only placement and comparison actions whose time complexity is less than $\Theta(n \log n)$ and returns `C` that is a permutation of `B` such that (C, b) is big-sorted

Answer 1.0.1. we assume in contradiction that the algorithm SORT-BIG which returns a big-sorted array runs in less than $\Theta(n \log n)$ time
consider the function:

```
MY-SORT(A):
    return SORT-BIG(A, -∞)
```

we know this function sorts a given array because SORT-BIG would sort the entire array as $-\infty$ is smaller than any number that would exist in the array and we know the function SORT-BIG sorts all numbers than are less than what is passed to it as an argument
the function MY-SORT is made up of one operation that calls the function SORT-BIG which we've assumed runs in less than $\Theta(n \log n)$ time, and so the function MY-SORT as a whole runs in less than $\Theta(n \log n)$ time which contradicts with the theorem that comparison-based sorting algorithms cannot run in less than $\Theta(n \log n)$ time and since we've arrived at a contradiction we know the time complexity of the function SORT-BIG has to have the boundary $\Omega(n \log n)$

Question 2. an array is "average-sorted" iff the average of every pair of numbers that are next to each other makes for an ascending sequence
example: the array $1, 10, 3, 20, 7, 22, 9$ is average-sorted
prove that there isnt a comparison-based algorithm that receives an array of integers and returns a permutation that is average-sorted in less than $\Theta(n \log n)$ operations

Answer 2.0.1. assume in contradiction that a function `Average-Sort` runs in less than $\Theta(n \log n)$ time
from the definition of "average-sorted" we can derive the following:

$$\frac{A[i] + A[i+1]}{2} > \frac{A[i+1] + A[i+2]}{2} \implies A[i+2] > A[i]$$

which means that its possible to split the array into 2 sorted subarrays, one from the odd indicies and the other from the even indicies, and then we can merge those two subarrays to form a sorted permutation of the original array:

```
My-Sort(A):
    B ← Average-Sort(A)
    even ← ∅
    odd ← ∅
    for i ← 1 to N do
        if i is even then
            add B[i] to vector even
        else
            add B[i] to vector odd
    return Merge(even, odd)
```

we know that My-Sort sorts a given array because Average-Sort would sort the array in a way so that the items at odd/even indicies form a sorted subarray on their own, and then we merge those two subarrays into one and return the result which is a sorted permutation of the original array
the second line calls the function Average-Sort which we've assumed runs in less than $\Theta(n \log n)$ time, the next 2 lines run in constant time, the for loop runs in $\Theta(n)$ time, and the call to Merge runs in $\Theta(n)$, so the time complexity of this sorting function is less than $\Theta(n \log n)$ as a whole, which contradicts with the theorem that every comparison-based sorting algorithm must run in atleast $\Theta(n \log n)$ time
since we've arrived at a contradiction we know the time complexity of the function Average-Sort must be bound from below by $\Theta(n \log n)$, i.e. $\Omega(n \log n)$

Question 3. an array is called "pair-sorted" if every pair of elements that are next to each other where the first is at an odd index preserve the following properties:

1. both elements differ

2. the bigger element of every pair form an ascending sequence

for example $3, 9, 32, 4, 40, 5, 56, 10, 7, 100$ is pair-sorted
prove/disprove that there is no algorithm that runs in less than $\Theta(n \log n)$ that receives an array of integers and returns a pair-sorted permutation of it

Answer 3.0.1. this is a proof by contradiction
assume in contradiction that the function Pair-Sort runs in less than $\Theta(n \log n)$ time and returns a pair-sorted permutation of the array it receives
consider the following algorithm:

```
My-Sort(A⃗):
    B⃗ ← Pair-Sort(A⃗)
    v⃗1 ← ∅
    v⃗2 ← ∅
    while i < |B⃗| do
        if B⃗[i] > B⃗[i+1] then
            add B⃗[i] to v⃗1
            B⃗[i] = −∞
        else
            add B⃗[i+1] to v⃗1
            B⃗[i+1] = −∞
        i ← i + 2
    i ← 1
    B⃗ ← Pair-Sort(B⃗)
    while i < |B⃗| do
        if B⃗[i] > B⃗[i+1] then
            add B⃗[i] to v⃗2
        else
            add B⃗[i+1] to v⃗2
        i ← i + 2
    return Merge(v⃗1, v⃗2)
```

this algorithm iterates twice through the array it receives, each loop is $\Theta(n)$
before each iteration, the algorithm runs Pair-Sort on the vector
on the first iteration the algorithm replaces all elements of the ascending sequence of numbers (that were made so by Pair-Sort) that are in each pair with $-\infty$ so that the second call to Pair-Sort would sort the elements that werent sorted in the first call because they were smaller than their partner
and by merging the two vectors we generated we'd get a sorted vector and we'd have accomplished that in less than $\Theta(n \log n)$ time because no operation or loop runs longer than $\Theta(n \log n)$, but this contradicts with the theorem that no comparison-based sorting function can sort in less than $\Theta(n \log n)$
and since we've arrived at a contradiction we know it cannot be possible that Pair-Sort runs in less than $\Omega(n \log n)$ time, so it has to be bound by it

Question 4. give a permutation of size `n` for whom after each i'th call to Partition one of the destination arrays would contain only one element

> **note**
> **Answer 4.0.1.** this answer assumes an implementation of quick sort that takes the first element as pivot

on each call to partition, two arrays are generated for pivot $A[i]$, for one of them to be of size 1 either all but one have to be bigger than $A[i]$, or all but one have to be smaller than $A[i]$, and since we'd be moving forward, im gonna pick the second option and see where that leads me
if $A[j]$ for all $j > i+1$ have to be smaller than $A[i]$, then we know $A[i+1]$, which would be the next pivot, has to be smaller than it
because we're partitioning an array to arrays of size 1 and $n-1$ its as if we're moving to the two elements at a time since the second to first element would be in its own partition and requires no further operations
so the sequence should be like: $\{x, > x, y, > y \wedge < x, z, > z \wedge < y, \dots\}$ such that $x > y > z > \dots$
so i came up with an appropriate example using these facts: $\{20, 22, 17, 18, 10, 15\}$

> **note**
> after looking at this example im realizing that this kind of permutations is just built up of two descending sequences, the first in the odd indicies and the second in the even indicies

Question 5. an array `A` of size `n` is called ascending-three iff a third of the array that contains the biggest elements in the array is sorted in ascending order and are placed in indicies that when divided by 3 gives a remainder of 1
i.e. $A[1] \leq A[4] \leq A[7] \leq A[10]$ and for each k such that $k \bmod 3 \neq 1$ we know $A[k] \leq A[1]$
prove/disprove that there is no algorithm that run in less than $\Theta(n \log n)$ that receives an array of integers and returns an ascending-three permutation of it

Answer 5.0.1. this is a proof by contradiction
assume in contradiction that the function Ascending-Three-Sort runs in less than $\Theta(n \log n)$ time and returns an ascending-three permutation of the array it receives
consider the following algorithm:

```
My-Sort(A⃗):
    B⃗ ← Ascending-Three-Sort(A⃗)
    v⃗1 ← Extract-Sorted(B⃗)
    B⃗ ← Ascending-Three-Sort(B⃗)
    v⃗2 ← Extract-Sorted(B⃗)
    B⃗ ← Ascending-Three-Sort(B⃗)
    v⃗3 ← Extract-Sorted(B⃗)
    return Merge(Merge(v⃗1, v⃗2), v⃗3)
Extract-Sorted(A⃗):
    B⃗ ← ∅
    for i ← 1 to |A⃗| do
        if i mod 3 = 1 then
            add A⃗[i] to vector B⃗
            A⃗[i] ← −∞
    return B⃗
```

we apply the algorithm Ascending-Three-Sort 3 times and on each time we extract the scattered subarray and fill its indicies with $-\infty$ to give way for other elements to be sorted, and at the end we merge those subarrays to get a sorted array, this is done in less than $\Theta(n \log n)$ time which contradicts with the theorem that any comparison-based algorithm cannot run in less than $\Omega(n \log n)$
and since we've arrived at a contradiction we know it cannot be possible that Ascending-Three-Sort runs in less than $\Omega(n \log n)$ time, so it has to be bound by it